Tutorial Técnico — `collections.deque`

1. Introdução

`deque` (*Double-Ended Queue*) é uma estrutura de dados do módulo `collections` do Python, projetada para realizar inserções e remoções eficientes em ambas as extremidades de uma sequência. É adequada para filas, pilhas e sistemas que necessitam manipular elementos no início e no final.

```
from collections import deque

fila = deque()
```

2. Arquitetura e funcionamento

No CPython, o `deque` utiliza uma organização interna baseada em **blocos de memória**, permitindo adicionar e remover elementos nas extremidades sem deslocar todos os demais elementos, como pode ocorrer em uma lista.

```
INÍCIO                                    FIM
  ↓                                        ↓
[ bloco ] ⇄ [ bloco ] ⇄ [ bloco ] ⇄ [ bloco ]
  ↑                                        ↑
appendleft()                            append()
popleft()                                pop()
```

Essa arquitetura é otimizada para operações nas pontas da estrutura, sendo uma das principais diferenças em relação à `list`.

3. Principais operações

Operação	Função	Big O
`append(x)`	adiciona no final	O(1)
`appendleft(x)`	adiciona no início	O(1)
`pop()`	remove do final	O(1)
`popleft()`	remove do início	O(1)

Exemplo:

```
fila.append("João")
fila.appendleft("Maria")
cliente = fila.popleft()
```

Nesse caso, `"Maria"` será o primeiro elemento removido.

4. Outras operações da API

- `extend(iterable)`: adiciona vários elementos no final — **O(k)**.
- `extendleft(iterable)`: adiciona vários elementos no início — **O(k)**.
- `clear()`: remove todos os elementos — **O(n)**.
- `copy()`: cria uma cópia superficial — **O(n)**.
- `count(x)`: conta ocorrências — **O(n)**.
- `index(x)`: retorna a posição de um elemento — **O(n)**.
- `rotate(n)`: desloca os elementos para a direita ou esquerda.
- `reverse()`: inverte a ordem — **O(n)**.
- `len(fila)`: retorna a quantidade de elementos.

5. Big O e eficiência

A notação **Big O** representa como o custo de uma operação cresce conforme aumenta a quantidade de elementos (`n`). No `deque` , `append` , `appendleft` , `pop` e `popleft` possuem complexidade **O(1)**, pois são operações realizadas nas extremidades.

Em uma lista, `list.pop(0)` possui complexidade **O(n)**, pois pode exigir o deslocamento dos demais elementos. Assim, o `deque` é mais eficiente quando inserções e remoções no início e no final são frequentes.

6. Acesso e escolha da estrutura

O `deque` permite acesso por índice, como `fila[0]` e `fila[-1]` , mas não é otimizado para acessos frequentes a posições intermediárias. Para esse tipo de operação, uma `list` pode ser mais adequada.

Característica	`list`	`deque`
Acesso por índice	Mais adequado	Menos adequado
Inserção no início	O(n)	O(1)
Remoção no início	O(n)	O(1)
Inserção no final	O(1)*	O(1)
Remoção no final	O(1)	O(1)
Operações nas duas pontas	Menos eficiente	Otimizado

*O(1) amortizado.